

PATRÓN DE DISEÑO: PROXY DE CONTEO

Capítulo 26 — Patrones de Diseño en Java

Traducción técnica al español

© 2004 CRC Press LLC — Traducción al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

El patrón Proxy de Conteo resulta útil para diseñar un conjunto de operaciones adicionales, como el registro de eventos (logging) y el conteo de invocaciones, que deben ejecutarse antes o después de que un objeto cliente invoque un método sobre un objeto proveedor de servicios. En lugar de mantener la implementación de estas operaciones adicionales dentro del objeto proveedor de servicios, el patrón Proxy de Conteo propone encapsularlas en un objeto separado, denominado proxy de conteo.

Una de las características de un objeto bien diseñado es que ofrece una funcionalidad enfocada. En otras palabras, un objeto, idealmente, no debería encargarse de tareas diversas no relacionadas entre sí. Encapsular las funcionalidades de registro, conteo y otras similares en un objeto separado deja al objeto proveedor de servicios únicamente con las responsabilidades para las que fue diseñado. Dicho de otra forma, permite que el proveedor de servicios realice una tarea bien definida y concreta.

Un proxy de conteo se diseña para tener la misma interfaz que el objeto proveedor de servicios al que accede el cliente. En lugar de acceder directamente al proveedor de servicios, los objetos cliente invocan los métodos sobre el proxy de conteo. El proxy realiza las operaciones de registro y conteo necesarias y reenvía la llamada al método al objeto proveedor de servicios (Figura 1).

El siguiente ejemplo ilustra cómo puede emplearse un proxy de conteo en un escenario de aplicación real.

Figura 1 — Asociación Genérica de Clases al Aplicar el Patrón Proxy de Conteo

```
Cliente —> ProxyConteo —[reenvía llamadas]—> ProveedorServicio
Todos implementan: <<interfaz>> InterfazComun { servicio() }
ProxyConteo también ejecuta: -operacionRegistro() -operacionConteo()
```

2. Ejemplo

Diseñemos una jerarquía de clases para pedidos tal como se muestra en la Figura 2. La interfaz PedidoIF declara un único método obtenerTodosPedidos para leer todos los pedidos desde un archivo.

Interfaz PedidoIF

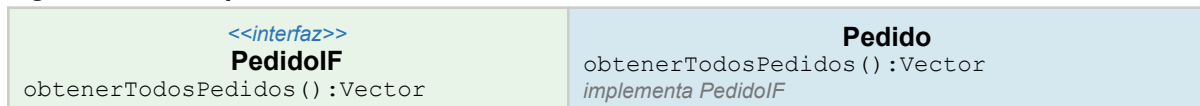
```
public interface PedidoIF {
    public Vector obtenerTodosPedidos();
}
```

Como parte de su implementación del método `obtenerTodosPedidos`, la clase `Pedido` hace uso de la clase utilitaria `UtilArchivo` para leer los elementos de pedido desde el archivo `pedidos.txt`.

Clase `Pedido`

```
public class Pedido implements PedidoIF {
    public Vector obtenerTodosPedidos() {
        UtilArchivo utilArchivo = new UtilArchivo();
        Vector v = utilArchivo.archivoAVector("pedidos.txt");
        return v;
    }
}
```

Figura 2 — Jerarquía de Clases de `Pedido`



Supongamos que se necesita registrar el tiempo que tarda en leerse el archivo de datos y la cantidad de veces que se invoca la operación `obtenerTodosPedidos`. Esta funcionalidad adicional puede diseñarse como una clase separada `ProxyPedido` que implemente la misma interfaz `PedidoIF` que la clase `Pedido` real. Esto garantiza que el `ProxyPedido` ofrezca la misma interfaz a los objetos cliente que el objeto `Pedido` real (Figura 3).

Clase `ProxyPedido` (Proxy de conteo)

```
public class ProxyPedido implements PedidoIF {
    private int contador = 0;

    public Vector obtenerTodosPedidos() {
        Pedido pedido = new Pedido();
        contador++;

        long t1 = System.currentTimeMillis();
        Vector v = pedido.obtenerTodosPedidos();
        long t2 = System.currentTimeMillis();

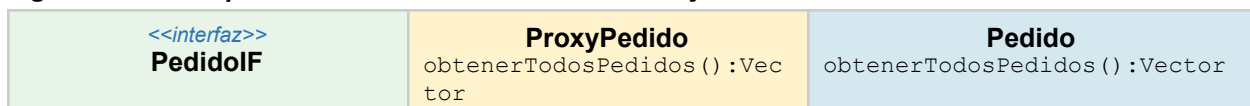
        long diferenciaTiempo = t2 - t1;
        String mensaje =
            "Iteracion=" + contador + "::Tiempo=" + diferenciaTiempo + "ms";

        // Registrar el mensaje en el archivo de log
        UtilArchivo utilArchivo = new UtilArchivo();
        utilArchivo.escribirEnArchivo("log.txt", mensaje, true, true);

        return v;
    }
}
```

El objeto cliente `AplicacionPrincipal` puede hacer uso del objeto `ProxyPedido` como si fuera el objeto `Pedido` real e invocar el método `obtenerTodosPedidos` de la interfaz `PedidoIF` sobre él. El `ProxyPedido` reenvía la llamada al objeto `Pedido` real, calcula el tiempo que tarda en leer todos los pedidos y registra estos detalles en un archivo de log mediante la clase auxiliar `UtilArchivo`. En este proceso, el `ProxyPedido` actúa como proxy de conteo.

Figura 3 — Jerarquía de Clases de `Pedido` con el Proxy de conteo



```
obtenerTodosPedidos():Vec  
tor
```

```
contador:int  
—— reenvía llamadas ——→
```

Clase Cliente AplicacionPrincipal

```
public class AplicacionPrincipal {  
    public static void main(String[] args) {  
        PedidoIF pedido = new ProxyPedido();  
  
        Vector v = pedido.obtenerTodosPedidos();  
        v = pedido.obtenerTodosPedidos();  
        v = pedido.obtenerTodosPedidos();  
        v = pedido.obtenerTodosPedidos();  
    }  
}
```

El objeto cliente `AplicacionPrincipal` declara la variable `pedido` con el tipo de la interfaz `PedidoIF` y la instancia con un objeto `ProxyPedido`. De este modo, el cliente interactúa con el proxy a través de la interfaz común, sin tener conocimiento de que sus llamadas son interceptadas, contadas y registradas antes de ser reenviadas al objeto `Pedido` real.

3. Preguntas de Práctica

1. Diseñe un proxy de conteo que lleve el registro del número de pedidos realizados y proporcione el monto promedio de los pedidos.
2. Considere los elementos de una biblioteca. Los elementos de la biblioteca pueden clasificarse en cuatro categorías: revistas, libros, videos y DVDs. Diseñe un proxy que lleve el conteo del número de elementos de cada categoría que son prestados cada día.